AST AUTOMATION

IMPLEMENTATION PLAN v2.0

RPA & No-Credential Approach | 6 Clients | 17 Automations

Version 2.0 | March 2026 | Confidential

6	17	2	0
Clients	Automations	Phases	Client Credentials Required

Executive Summary

This document is the updated automation implementation plan for six AST client operations. All automation approaches have been revised to use RPA (Robotic Process Automation) and browser-session-based methods — eliminating the need to request API credentials, backend access, or system integrations from any client.

The core principle: agents are already logged into every system they use (SF, CTM, billing portals, trackers). All automations work on top of these active sessions using browser extensions, screen scraping, DOM interaction, local scripts, and simulated input — operating entirely on data that is already visible on the agent's screen or machine.

RPA-First Architecture Principles

- No API credentials, tokens, or backend access required from any client
- Works on existing agent login sessions — piggybacks on authenticated browsers
- All NLP/AI processing runs locally on the agent's machine — no cloud data exposure
- Config files (JSON) allow team leads to update rules, templates, and mappings without developer help
- Browser extensions (Chrome/Edge) + desktop Python scripts = zero server infrastructure required
- Fallback to manual always available — automation assists, never blocks the agent

Core RPA Toolstack

Tool / Library	Use Case	Why No Credentials Needed
Playwright / Selenium	Multi-tab browser automation, form filling, navigation	Uses agent's existing authenticated browser session
Chrome Extension (Content Scripts)	DOM reading, overlay injection, event detection	Runs inside the browser, reads page content directly
PyAutoGUI / AutoIt	Desktop-level clicks, keystrokes, hotkey triggers	Simulates user input — no system access required
Tesseract OCR	Read data from screens, PDFs, portal images	Purely local image processing, offline
Python watchdog	Monitor Downloads folder for CSV/PDF exports	File system watcher, no network access needed
pdfplumber / pdfrw	Read and write PDF form fields	Local file manipulation, no PDF service needed
DistilBERT / spaCy (local)	Note summarization, call classification	Model runs on-device — no API calls, no data sent externally
LibreTranslate (offline)	Non-English call/note translation	Self-hosted local instance, no cloud translation API

Phased Rollout Overview

Phase	Timeline	Focus Areas	Clients
Phase 1	Foundation &	CTM-SF session-based pop-ups, auto-note	All 6 clients

Phase	Timeline	Focus Areas	Clients
(Weeks 1–6)	Quick Wins	generators, wrap-up multi-tab sync, lead pruning via CSV watcher, auto-fax via portal RPA, PDF auto-filler	
Phase 2 (Weeks 7–14)	Intelligence Layer	AI reply/note suggesters (local NLP), queue prioritizer overlay, negotiation form-filler, portal scraping with cache, template generator app	All 6 clients
Phase 3 (Weeks 15–20)	Tuning & Expansion	Model accuracy tuning, new portal profiles, team-lead config UIs, field-mapping updates, user feedback loops	Legacy, Banyan, TBT

Flyland Recovery

Addiction counseling center facing manual SF lookups, note duplication between CTM and SF, and 1–2 hour transfer waits to US reps. Specialists handle 50–100% of shifts on calls/chats across night/morning/mid shifts.

Pain Points

- Manual SF account lookups take 1–3 mins per call with slow load times
- Notes duplicated across both CTM and SF systems post-call
- Transfer waits to US reps up to 2 hours (10–20 min queue)
- Repetitive note-typing for unqualified or duplicate callers
- Manual chat replies typed individually per interaction

Automation Summary

Automation	Priority	Effort	Phase	Business Impact
CTM-SF Auto-Account Access	HIGH	Medium	Phase 1	Saves 1–3 mins per call; zero credential exposure
Auto-Note Generator for Unqualified/Duplicates	HIGH	Medium	Phase 1	Eliminates 1–2 mins of note typing per unqualified/duplicate call
Chat Reply Suggester & Form Auto-Filler	MEDIUM	Low-Medium	Phase 2	Reduces 1–3 mins chat writing + 1 min form fill per interaction

Automation Detail Specs

CTM-SF Auto-Account Access

Priority	HIGH
Phase	Phase 1
Effort Level	Medium
Inputs	Caller phone number visible on CTM screen
Outputs	SF account auto-opened in adjacent browser tab/panel
Technical Approach	Browser extension (Chrome/Edge) detects incoming call UI in CTM via DOM monitoring. Extracts caller number from CTM's call notification element. Auto-opens SF search URL in a side panel pre-filled with the number — no credentials needed, agent is already logged into SF. Extension reads the resulting SF page and surfaces key fields (name, history, last contact) in a floating overlay.
Edge Cases	No SF match: extension shows 'New Lead' button that pre-fills SF new lead form with phone number. Slow SF page: timeout with manual fallback notice.
Testing Criteria	Record/replay testing via Playwright; verify panel appears within 2s of call notification.
Business Impact	Saves 1–3 mins per call; zero credential exposure

⚙️ RPA / No-Credential Approach

- Chrome Extension with content script monitors CTM DOM for call events
- Playwright/Selenium script triggered on call-start reads the phone number from screen
- Opens SF in existing browser session using pre-built search URL pattern
- Overlay panel injects into browser — reads SF DOM, no SF API required
- Duplicate detection: scans SF search results page for multiple matches and alerts agent

Auto-Note Generator for Unqualified/Duplicates

Priority	HIGH
Phase	Phase 1
Effort Level	Medium
Inputs	Call transcript text (from screen/clipboard), disposition selected in CTM
Outputs	Draft note pre-filled in SF/CTM note field
Technical Approach	Desktop listener monitors CTM's disposition dropdown selection. On 'Unqualified' or 'Duplicate' selection, reads any transcript text visible on screen via DOM scraping or clipboard capture. Runs local NLP (spaCy/transformers offline model) to classify and generate summary note. Injects generated note text directly into the SF/CTM notes field via simulated keystrokes (PyAutoGUI) or DOM injection.
Edge Cases	Complex or emotional content: skip auto-note, flag for manual. Low transcript confidence: show draft with 'Review before posting' prompt.
Testing Criteria	Replay recorded dispositions; validate 80% of drafts accepted without edits.
Business Impact	Eliminates 1–2 mins of note typing per unqualified/duplicate call

⚙️ RPA / No-Credential Approach

- PyAutoGUI or Playwright detects disposition field change in CTM
- Screen region OCR (Tesseract) or clipboard monitor captures transcript text
- Local NLP model (spaCy or lightweight BERT) classifies call type offline — no cloud API
- Generated note injected into note field via DOM manipulation or simulated typing
- Empathy flags preserved: keywords like 'crisis', 'distress' skip automation and alert agent

Chat Reply Suggester & Form Auto-Filler

Priority	MEDIUM
Phase	Phase 2
Effort Level	Low-Medium
Inputs	Active chat window content (screen), form type visible on screen
Outputs	Suggested replies in floating panel; form fields populated
Technical Approach	Chrome extension reads chat window DOM to extract conversation context. Matches keywords against a local template library (JSON file on agent's machine) to surface 2–3 reply suggestions in a non-intrusive sidebar. For admission forms, detects form type by page URL or field labels and auto-fills

	using data already visible in the open SF record.
Edge Cases	Long chats: summarize last 5 messages only for context. Unrecognized form: skip auto-fill, show manual prompt.
Testing Criteria	Measure reply selection rate and time-to-send before/after.
Business Impact	Reduces 1–3 mins chat writing + 1 min form fill per interaction

⚙️ RPA / No-Credential Approach

- Chrome content script reads chat DOM in real-time for keyword matching
- Local JSON template library — no external service needed
- Form filler reads field labels via DOM, matches to SF data already on screen
- One-click 'Insert' button pastes suggestion into chat input field
- Offline mode: templates cached locally, works without internet connectivity

Legacy

Multi-channel contact center with post-call duplication across CTM, SF, chatter, and separate trackers. Agents spend significant time re-entering identical data across systems after every interaction.

Pain Points

- Same disposition entered manually into CTM, SF, chatter, and tracker
- Customer history requires manual SF lookup per call (2–5 mins, slow load)
- Repetitive form fills and tagging after every interaction
- Long US rep approval queues (3 mins to 1 hour)
- Repeat callers re-entered by different agents

Automation Summary

Automation	Priority	Effort	Phase	Business Impact
Wrap-Up Auto-Sync Across Systems	HIGH	High	Phase 1	Eliminates 1–2 hrs/shift of redundant data entry
Customer History Auto-Pull & Reminders	HIGH	Medium	Phase 1	Saves 2–5 mins of manual lookup per call
Form-Filler, Note Summarizer & AI Queue Prioritizer	MEDIUM	High	Phase 2	Reduces wrap-up and triage time by an estimated 30–50%

Automation Detail Specs

Wrap-Up Auto-Sync Across Systems

Priority	HIGH
Phase	Phase 1
Effort Level	High
Inputs	Wrap-up form data visible on screen after call ends
Outputs	Same data automatically entered into all other open system tabs
Technical Approach	After agent completes wrap-up in primary system (CTM), a desktop automation script reads the submitted form data from the confirmation screen using DOM scraping. It then switches to each other open browser tab (SF, chatter, tracker) and fills matching fields via simulated input. Uses a field-mapping config file (maintained locally) to match field names across systems. No credentials needed — agent is already logged into all tabs.
Edge Cases	Tab not open: skip with alert to agent. Conflicting existing data: highlight and ask agent before overwriting.
Testing Criteria	End-to-end simulation across all 4 systems; verify zero duplicate entries.
Business Impact	Eliminates 1–2 hrs/shift of redundant data entry

⚙️ **RPA / No-Credential Approach**

- Playwright multi-tab controller: identifies open tabs by URL pattern
- Reads wrap-up data from CTM confirmation page DOM
- Field mapping config (local JSON) maps CTM field names to SF/chatter/tracker equivalents
- Simulated typing + tab navigation fills each system sequentially
- Duplicate detection: checks if record already exists by scanning page before writing

Customer History Auto-Pull & Reminders

Priority	HIGH
Phase	Phase 1
Effort Level	Medium
Inputs	Caller ID visible in CTM call notification
Outputs	SF history panel auto-opened; reminder pop-ups on screen
Technical Approach	Chrome extension detects incoming call in CTM DOM. Extracts phone number and auto-navigates SF (already open) to the caller's record using SF's built-in search. Reads resulting page data and builds a lightweight history summary card injected as an overlay. Rule-based reminder engine (local config) checks for conditions like 'unqualified' status and surfaces pop-up reminders.
Edge Cases	Slow SF load: show progress bar and partial results. Sensitive fields: audit log entry written to local file.
Testing Criteria	Validate reminders fire correctly for all configured rule conditions.
Business Impact	Saves 2–5 mins of manual lookup per call

⚙️ RPA / No-Credential Approach

- Content script polls CTM DOM for active call state changes
- Auto-navigates SF search using extracted phone number — uses existing login session
- Overlay card reads SF DOM fields: name, last contact date, status, notes
- Local rules engine (JSON config): maps SF status values to reminder messages
- Progress indicator shown during SF page load; partial data displayed immediately

Form-Filler, Note Summarizer & AI Queue Prioritizer

Priority	MEDIUM
Phase	Phase 2
Effort Level	High
Inputs	Queue list visible in CTM, notes text on screen, form fields on active page
Outputs	Sorted queue display overlay, note summaries, auto-filled form fields
Technical Approach	Three coordinated components: (1) Form filler reads active form's field labels via DOM and fills from data visible in open SF tab. (2) Note summarizer runs locally using a lightweight transformer model (e.g., DistilBERT) on note text copied to clipboard — outputs summary under 100 words injected back. (3) Queue prioritizer reads CTM queue list via DOM, scores items using keyword

	rules (crisis terms = high urgency), re-renders a sorted visual overlay on top of CTM.
Edge Cases	Queue > 10: cap display at top 10 by urgency. Abstract content: fallback to full text display.
Testing Criteria	Verify complex/crisis cases consistently rank at top of queue.
Business Impact	Reduces wrap-up and triage time by an estimated 30–50%

⚙️ RPA / No-Credential Approach

- DOM-based form filler with local field-mapping config file
- Clipboard hook captures note text; local transformer model summarizes offline
- CTM queue DOM scraper reads caller info, wait time, and any visible notes
- Urgency scoring via local keyword list (configurable JSON)
- Queue overlay rendered as floating browser panel — does not modify CTM data

TBT

Outbound-heavy operation (40–50% OB calls) dealing with old FB form leads and hourly dial cycles. ASTs spend half their shift dialing unresponsive leads and manually managing SF/CTM updates.

Pain Points

- Hourly outbound dials to leads with no response for weeks or months
- SF/CTM duplication across notes, trackers, VOB records
- Inbound routing lacks equal load balancing
- SMS fallback to voicemail managed manually
- Insurance verification is manual with no status checker

Automation Summary

Automation	Priority	Effort	Phase	Business Impact
Auto-Lead Pruning & Dial/SMS Automation	HIGH	Medium	Phase 1	Saves 2–3 hrs/shift on dead-lead outbound cycles
Auto-Lead Pull & Inbound Routing	HIGH	Medium	Phase 1	Eliminates manual routing decisions and lead lookup overhead
Text Reply Suggester, Note Templates & Insurance Status Checker	MEDIUM	Medium	Phase 2	Reduces note-writing and insurance verification by 3–5 mins per call

Automation Detail Specs

Auto-Lead Pruning & Dial/SMS Automation

Priority	HIGH
Phase	Phase 1
Effort Level	Medium
Inputs	Lead list exported from SF/CTM as CSV (standard export agents already do)
Outputs	Filtered lead list, auto-dial sequence, SMS drafts queued
Technical Approach	Agent exports lead list from SF/CTM as CSV (existing workflow). A local Python script watches the Downloads folder for new CSV exports, parses them automatically, and filters out leads with last_contact older than threshold (configurable, default 14 days). Cleaned list is loaded into CTM's predictive dialer by simulating the import UI. For voicemail-detected calls, SMS templates are auto-drafted and queued in the messaging tool using DOM injection.
Edge Cases	Partial responses (e.g., 1 text back): flag lead, exclude from prune. Bulk prune: batch in chunks of 100.
Testing Criteria	Simulate 200-lead list; verify 50%+ reduction in dialer queue.
Business Impact	Saves 2–3 hrs/shift on dead-lead outbound cycles

⚙️ RPA / No-Credential Approach

- Local file watcher (Python watchdog) monitors Downloads folder for SF/CTM CSV exports
- Pandas-based CSV parser applies prune rules — runs fully offline
- Playwright bot imports filtered list into CTM dialer via the existing import UI
- Voicemail detection: monitors CTM call result field; on VM result, opens SMS tool and pre-fills template
- Batch limit: 100 leads per run to avoid overwhelming the dialer

Auto-Lead Pull & Inbound Routing

Priority	HIGH
Phase	Phase 1
Effort Level	Medium
Inputs	Incoming call notification visible in CTM, agent status visible on screen
Outputs	Lead data pre-loaded in SF; call routed to next available agent
Technical Approach	Chrome extension reads incoming call data from CTM notification DOM. Simultaneously reads agent status panel (also in CTM DOM) to identify available agents. Round-robin state maintained in a local JSON file on the machine. Extension auto-opens matching SF lead in a side panel. Routing suggestion displayed to supervisor as an overlay — no backend change needed.
Edge Cases	No lead found: show 'Create Lead' shortcut. All agents busy: display wait queue with ETA.
Testing Criteria	Simulate 20 calls; verify even distribution across 5 agents.
Business Impact	Eliminates manual routing decisions and lead lookup overhead

⚙️ RPA / No-Credential Approach

- Content script reads CTM call queue and agent status DOM in real-time
- Round-robin state stored in localStorage or local JSON file — no backend needed
- SF lead lookup triggered by phone match using existing browser session
- Routing overlay shows supervisor which agent to assign next
- Unbalanced load alert: highlights agents above 2x average queue depth

Text Reply Suggester, Note Templates & Insurance Status Checker

Priority	MEDIUM
Phase	Phase 2
Effort Level	Medium
Inputs	Active SMS/chat window, note field on screen, insurance info visible in SF
Outputs	Suggested texts, pre-filled note templates, qualified/unqualified badge
Technical Approach	Chrome extension reads SMS/chat content from DOM and matches against local template library for suggestions. Note templates triggered by disposition selection in CTM — pre-fills note field with structured template. Insurance

	checker reads insurance name and member ID from SF record currently open, then uses Playwright to auto-navigate to the relevant insurance portal (agent already logged in) and scrapes the eligibility/coverage status.
Edge Cases	Portal navigation fails: alert agent to check manually. Non-standard insurer: skip checker, show manual prompt.
Testing Criteria	Target > 85% correct status reads across top 10 insurance portals used.
Business Impact	Reduces note-writing and insurance verification by 3–5 mins per call

⚙️ RPA / No-Credential Approach

- DOM reader for SMS window content; local template JSON for suggestions
- Disposition-triggered note template injection via simulated keystrokes
- Playwright navigates to insurance portal in new tab using agent's existing login
- Screen scraper reads eligibility result and displays qualified/unqualified badge in overlay
- Ambiguous results: prompt agent for manual confirmation before tagging

Takahami

Medical billing and claims negotiation team dealing with daily carrier contacts, lengthy claim lookups, manual fax submissions for appeals, and repetitive status updates per Date of Service.

Pain Points

- Manual fax submission for insurance appeals (5+ mins per submission)
- Daily carrier contacts for same accounts (repetitive, 1–2 hr waits)
- 20-minute claim lookups with double-check verification
- Status changes (PAID → UNDERPAID) done manually per claim
- Notation duplication per Date of Service in reports

Automation Summary

Automation	Priority	Effort	Phase	Business Impact
Auto-Fax for Appeals & Claim Status Updater	HIGH	Medium	Phase 1	Saves 5–10 mins per appeal; eliminates manual fax navigation
Claim-Type-Based Appeal Router	HIGH	Low	Phase 1	Eliminates manual destination lookup per claim; reduces routing errors
Negotiation Form-Filler & Reply Suggester	MEDIUM	Medium	Phase 2	Reduces 10–15 mins of drafting per negotiation/appeal

Automation Detail Specs

Auto-Fax for Appeals & Claim Status Updater

Priority	HIGH
Phase	Phase 1
Effort Level	Medium
Inputs	Appeal document open on screen (PDF viewer or web form), claim record open in billing portal
Outputs	Fax queued and sent via online fax tool; claim status field updated
Technical Approach	Agent prepares appeal document as usual. RPA bot monitors for a 'Send Appeal' button click (or hotkey trigger). It captures the active PDF path from the browser download bar or file picker using screen OCR. Then it auto-navigates to the online fax portal the team already uses (agent is logged in), fills the recipient fax number (pulled from a local carrier fax directory JSON), attaches the file, and submits. Status update: bot reads current claim status in the billing portal DOM and changes it via UI interaction.
Edge Cases	Fax portal session expired: alert specialist to re-login. Invalid status transition: validation check from local rules file.
Testing Criteria	Full cycle simulation: prep doc → fax submission → status change → confirmation.
Business Impact	Saves 5–10 mins per appeal; eliminates manual fax navigation

⚙️ RPA / No-Credential Approach

- PyAutoGUI hotkey listener triggers fax workflow on specialist's command
- Screen OCR (Tesseract) identifies open PDF filename from title bar or download bar
- Playwright navigates existing fax portal session — no fax API credentials required
- Carrier fax directory: local JSON file mapping carrier names to fax numbers
- Claim status update: DOM interaction on billing portal — reads current value, selects new status from dropdown

Claim-Type-Based Appeal Router

Priority	HIGH
Phase	Phase 1
Effort Level	Low
Inputs	Claim record visible on screen in billing portal
Outputs	Routing decision shown as overlay (fax number or email address)
Technical Approach	Chrome extension reads claim type, plan name, and payer fields from the billing portal DOM. Applies a local rule engine (JSON config: claim type → destination) to determine whether the appeal goes to local plan fax, home plan fax, or email. Displays the correct destination prominently as an overlay and pre-fills the fax/email tool accordingly. Config file is editable by team lead — no developer needed for rule updates.
Edge Cases	Unknown type: default to manual with alert. Multi-plan claims: prompt agent to select plan.
Testing Criteria	Target 100% routing accuracy for all known claim types in config.
Business Impact	Eliminates manual destination lookup per claim; reduces routing errors

⚙️ RPA / No-Credential Approach

- Content script reads claim type and payer fields from billing portal DOM
- Local routing rules config (JSON): editable by team lead without code changes
- Overlay card shows routing decision with one-click 'Use This' button
- Pre-fills fax number or email address in the communication tool already open
- Unknown claim type: display 'Manual Review Required' with routing guide link

Negotiation Form-Filler & Reply Suggester

Priority	MEDIUM
Phase	Phase 2
Effort Level	Medium
Inputs	Carrier response visible on screen (email/portal message), negotiation form open
Outputs	Pre-filled negotiation form, suggested reply/next step in overlay
Technical Approach	Chrome extension reads carrier response text from the open email or portal

	message DOM. Runs local keyword matching against a negotiation playbook (JSON config) to identify response type (e.g., 'denial', 'partial approval', 'request for info'). Surfaces 2–3 suggested next steps/reply templates. For the negotiation form, reads existing claim data from the open billing portal tab and injects it into the negotiation form fields using DOM manipulation.
Edge Cases	Short/vague carrier response: default to general template. Complex negotiations: escalation button in overlay.
Testing Criteria	Survey-based time reduction target: 1–2 hrs saved per shift.
Business Impact	Reduces 10–15 mins of drafting per negotiation/appeal

⚙️ RPA / No-Credential Approach

- DOM reader captures carrier response text from email client or portal message thread
- Local negotiation playbook (JSON): response patterns mapped to suggested actions
- Overlay panel shows ranked suggestions with 'Copy' and 'Insert' buttons
- Form filler reads claim fields from open billing portal tab; injects into negotiation form
- Complex cases: 'Escalate' button in overlay opens team lead chat with pre-filled context

Banyan

Addiction counseling center with inbound call inefficiencies. Specialist transfer waits (5–10 mins), duplicate data between tracker and SF, and limited caller info causing admission ticket delays.

Pain Points

- Manual SF lookup takes 1–2 mins per call with system latency
- Admission tickets created with limited caller info (70% of time)
- Transfer waits to specialist via group chat: 5–10 mins, up to 40% of shift
- Post-call tracker and SF updated separately (duplication every call)
- Repeat callers re-entered manually in SF

Automation Summary

Automation	Priority	Effort	Phase	Business Impact
CTM-SF Auto-Account Pop-Up	HIGH	Medium	Phase 1	Saves 1–2 mins per call; eliminates most common manual lookup
Auto-Call Tracking & Form/Note Auto-Fill	HIGH	High	Phase 1	Eliminates duplicate entry; saves 1–2 mins of wrap-up per call
Note Summarizer & Reply Suggester	MEDIUM	Medium	Phase 2	Reduces review and response time; speeds up admissions workflow

Automation Detail Specs

CTM-SF Auto-Account Pop-Up

Priority	HIGH
Phase	Phase 1
Effort Level	Medium
Inputs	Caller phone number visible in CTM call notification
Outputs	SF account data displayed as floating overlay; new lead shortcut if no match
Technical Approach	Chrome extension with content script monitors CTM tab for incoming call events by polling the DOM. On call detection, extracts caller phone number from CTM's call notification element. Uses the phone number to construct a SF search URL and opens it in the background within the same browser window (agent already logged into SF). Reads the resulting SF page DOM to extract key fields (name, admission history, last contact, status) and renders them as a floating overlay card pinned to the corner of the screen.
Edge Cases	SF timeout after 5s: overlay shows partial data with 'Open SF manually' link. Duplicate accounts: prompt merge.
Testing Criteria	Simulate 50 calls; verify overlay renders within 2s for 90% of cases.
Business Impact	Saves 1–2 mins per call; eliminates most common manual lookup

⚙️ RPA / No-Credential Approach

- Content script polls CTM DOM for call state changes every 500ms
- Phone number extracted via DOM selector targeting CTM's caller ID element
- SF opened in background tab using search URL pattern — no API token needed
- Overlay card reads SF DOM fields and renders summary using injected CSS/JS
- Duplicate accounts: scans SF results for multiple entries and shows merge prompt
- No match: 'Create Lead' button in overlay pre-fills SF new lead form with phone number

Auto-Call Tracking & Form/Note Auto-Fill

Priority	HIGH
Phase	Phase 1
Effort Level	High
Inputs	Call metadata visible in CTM post-call (duration, disposition, outcome), agent's wrap-up screen
Outputs	Tracker row added automatically; SF record updated; ticket pre-filled
Technical Approach	After agent selects disposition in CTM wrap-up, a Playwright script reads all visible wrap-up fields from CTM DOM. It switches to the tracker (Google Sheets or internal web tracker open in another tab) and adds a new row by simulating clicks and keystrokes — no Sheets API needed. Simultaneously navigates to the SF record (already open from the earlier pop-up) and updates relevant fields. Ticket pre-fill: opens admission ticket form and populates caller info fields from data already on screen.
Edge Cases	Incomplete transcription: show editable draft with required fields highlighted. Conflicting data: alert agent before overwriting.
Testing Criteria	End-to-end simulation; verify zero duplicate entries across tracker and SF.
Business Impact	Eliminates duplicate entry; saves 1–2 mins of wrap-up per call

⚙️ RPA / No-Credential Approach

- Playwright multi-tab manager: reads CTM wrap-up DOM after disposition submit
- Tracker update: navigates to tracker tab, uses keyboard shortcut to add row, fills via Tab/Enter simulation
- SF update: navigates to open SF record tab, fills fields via DOM injection or simulated input
- Ticket pre-fill: reads SF record data already on screen, injects into ticket form fields
- Repeat caller detection: SF search checks for existing record before creating new entry

Note Summarizer & Reply Suggester

Priority	MEDIUM
Phase	Phase 2
Effort Level	Medium
Inputs	Note text visible on screen or copied to clipboard, chat/reply window content
Outputs	Summary under 100 words shown in overlay; suggested reply options panel

Technical Approach	Agent selects (highlights) note text in SF or copies it — clipboard hook captures content. Local transformer model (DistilBERT or similar, running offline on agent machine) generates a summary under 100 words. Summary is displayed in a persistent overlay panel and can be inserted into fields with one click. Reply suggester reads the active chat/message window DOM, matches keywords to a local template DB, and surfaces 2–3 options in the same panel.
Edge Cases	Empty clipboard: show 'Select text first' prompt. Sensitive content flag: skip auto-summarize, alert supervisor.
Testing Criteria	User acceptance test: target 30% time reduction per survey.
Business Impact	Reduces review and response time; speeds up admissions workflow

⚙️ RPA / No-Credential Approach

- Clipboard monitor captures highlighted/copied text automatically
- Local NLP model runs offline — no cloud processing, no data leaves the machine
- Summary overlay panel: one-click insert into active note field
- Template DB: local JSON with keyword-to-reply mappings, updatable by team lead
- Non-English text: passed through local translation layer (LibreTranslate offline) before summarization

Element Medical Billing

VOB and verification specialists fill 10+ PDF forms daily, navigate multiple insurance portals, and spend up to 4 hours per shift in IVR/queue waits. Same patient data re-entered repeatedly across forms.

Pain Points

- 10+ PDF forms filled manually per day per specialist
- Same patient/insurance data re-entered across multiple forms
- 1–4 hour IVR/queue waits to reach insurance representatives
- Multiple systems updated post-call (duplication)
- New portals require manual navigation without API support

Automation Summary

Automation	Priority	Effort	Phase	Business Impact
PDF Auto-Filler	HIGH	Medium	Phase 1	Saves 2–5 mins × 10+ forms/day = 20–50+ mins daily per specialist
Insurance Portal Auto-Lookup & Reminders	HIGH	High	Phase 2	Eliminates manual portal navigation; surfaces missed fields automatically
Pre-Filled Insurance Form Template Generator	MEDIUM	Low	Phase 2	Eliminates template setup time; ensures field consistency across all forms

Automation Detail Specs

PDF Auto-Filler

Priority	HIGH
Phase	Phase 1
Effort Level	Medium
Inputs	Blank PDF form opened by specialist, patient record open in SF/billing system
Outputs	PDF with standard fields auto-populated; specialist fills only non-standard fields
Technical Approach	When specialist opens a PDF form, a desktop Python script (running as a background tray app) detects the new PDF via file watcher. It reads the PDF's field labels using pdfplumber (no server needed). Simultaneously reads patient data from the currently open SF/billing portal tab via DOM scraping. Matches field labels to patient data using a local field-mapping config. Writes data into PDF fields using PyPDF2/pdfrw and saves a pre-filled copy for the specialist to review and finalize.
Edge Cases	Missing patient data: highlight unfilled fields in yellow for specialist. Large PDFs: process in chunks. Unrecognized form: skip and alert.
Testing Criteria	Fill 10 representative forms; target 100% accuracy on repeat fields; time savings > 50%.
Business Impact	Saves 2–5 mins × 10+ forms/day = 20–50+ mins daily per specialist

⚙️ RPA / No-Credential Approach

- Python file watcher (watchdog) detects when specialist opens a new PDF
- pdfplumber reads form field labels offline — no external service needed
- DOM scraper reads patient data from open SF/billing portal tab in Chrome
- Local field-mapping config (JSON): maps PDF field labels to SF DOM selectors
- pdfrw/PyPDF2 writes values into PDF fields and saves pre-filled copy to same folder
- Preview UI: system notification with 'Open Filled PDF' button

Insurance Portal Auto-Lookup & Reminders

Priority	HIGH
Phase	Phase 2
Effort Level	High
Inputs	Insurance portal open in browser (agent already logged in), patient policy ID visible on screen
Outputs	Eligibility/coverage data extracted and displayed in overlay; reminder pop-ups for missed fields
Technical Approach	Chrome extension detects when specialist navigates to a known insurance portal (by URL pattern). Reads the policy ID from the open SF/billing record tab. Auto-navigates to the portal's eligibility lookup page and enters the policy ID via DOM interaction — specialist is already authenticated. Scrapes the result page for key coverage fields (deductible, copay, in/out-of-network status). Displays results in a floating overlay and triggers reminders for any missing/expiring data. Cached results stored in browser localStorage to avoid repeat lookups.
Edge Cases	Portal downtime: use cached result with timestamp shown. Auth expired: alert specialist to re-login. New portal: specialist adds URL pattern via settings panel.
Testing Criteria	Simulate lookups on top 10 insurance portals; target > 90% field accuracy.
Business Impact	Eliminates manual portal navigation; surfaces missed fields automatically

⚙️ RPA / No-Credential Approach

- Content script detects portal URL and activates portal-specific scraping profile
- Policy ID read from open SF/billing tab DOM automatically
- Playwright sub-script navigates portal eligibility form using existing authenticated session
- Result scraper extracts coverage fields from portal response page DOM
- Results cached in localStorage per policy ID with 24hr expiry
- Pop-up reminders triggered by local rules (e.g., 'Deductible field empty')

Pre-Filled Insurance Form Template Generator

Priority	MEDIUM
Phase	Phase 2
Effort Level	Low
Inputs	Insurance type selected by specialist (dropdown in local tool)

Outputs	Template PDF with common fields pre-filled, ready for specialist to complete
Technical Approach	A lightweight local desktop app (Tkinter or Electron) presents a simple dropdown of insurance types. On selection, it loads the matching template PDF from a local templates folder, pre-fills standard fields (insurance name, address, standard codes) from a local JSON config, and opens the result in the system PDF viewer. Specialists can upload new template PDFs and map their fields through a simple UI — no developer needed.
Edge Cases	Custom/non-standard insurer: 'Upload Template' option in app. Missing field values: highlight in yellow.
Testing Criteria	Generate templates for all active insurers; gather specialist feedback for iteration.
Business Impact	Eliminates template setup time; ensures field consistency across all forms

⚙️ RPA / No-Credential Approach

- Local desktop app (Tkinter) — no internet connection required
- Templates stored as PDF files in a shared network folder or local directory
- JSON config maps insurance type to standard field values
- Field mapping UI allows team lead to add new insurance types and templates
- Output: pre-filled PDF opened in default viewer, saved to specialist's working folder

Master Implementation Roadmap

Phase 1 — Foundation & Quick Wins (Weeks 1–6)

All Phase 1 automations use session-based browser automation and local file processing — deployable without any client backend access.

Automation	Client	Priority	Effort	Primary RPA Method
CTM-SF Auto-Account Access	Flyland Recovery	HIGH	Medium	Chrome Extension with content script monitors CTM DOM for call events
Auto-Note Generator for Unqualified/Duplicates	Flyland Recovery	HIGH	Medium	PyAutoGUI or Playwright detects disposition field change in CTM
Wrap-Up Auto-Sync Across Systems	Legacy	HIGH	High	Playwright multi-tab controller: identifies open tabs by URL pattern
Customer History Auto-Pull & Reminders	Legacy	HIGH	Medium	Content script polls CTM DOM for active call state changes
Auto-Lead Pruning & Dial/SMS Automation	TBT	HIGH	Medium	Local file watcher (Python watchdog) monitors Downloads folder for SF/CTM CSV exports
Auto-Lead Pull & Inbound Routing	TBT	HIGH	Medium	Content script reads CTM call queue and agent status DOM in real-time
Auto-Fax for Appeals & Claim Status Updater	Takahami	HIGH	Medium	PyAutoGUI hotkey listener triggers fax workflow on specialist's command
Claim-Type-Based Appeal Router	Takahami	HIGH	Low	Content script reads claim type and payer fields from billing portal DOM
CTM-SF Auto-Account Pop-Up	Banyan	HIGH	Medium	Content script polls CTM DOM for call state changes every 500ms
Auto-Call Tracking & Form/Note Auto-Fill	Banyan	HIGH	High	Playwright multi-tab manager: reads CTM wrap-up DOM after disposition submit
PDF Auto-Filler	Element Medical Billing	HIGH	Medium	Python file watcher (watchdog) detects when specialist opens a new PDF

Phase 2 — Intelligence Layer (Weeks 7–14)

Phase 2 adds local AI/NLP models, portal scraping with caching, and team-lead configurable tools. Still zero credential requirements.

Automation	Client	Priority	Effort	Primary RPA Method
Chat Reply Suggester & Form Auto-Filler	Flyland Recovery	MEDIUM	Low-Medium	Chrome content script reads chat DOM in real-time for keyword matching
Form-Filler, Note Summarizer & AI Queue Prioritizer	Legacy	MEDIUM	High	DOM-based form filler with local field-mapping config file
Text Reply Suggester, Note Templates & Insurance Status Checker	TBT	MEDIUM	Medium	DOM reader for SMS window content; local template JSON for suggestions
Negotiation Form-Filler & Reply Suggester	Takahami	MEDIUM	Medium	DOM reader captures carrier response text from email client or portal message thread
Note Summarizer & Reply Suggester	Banyan	MEDIUM	Medium	Clipboard monitor captures highlighted/copied text automatically
Insurance Portal Auto-Lookup & Reminders	Element Medical Billing	HIGH	High	Content script detects portal URL and activates portal-specific scraping profile
Pre-Filled Insurance Form Template Generator	Element Medical Billing	MEDIUM	Low	Local desktop app (Tkinter) — no internet connection required

Deployment Requirements (Per Agent Machine)

- Python 3.10+ with: playwright, selenium, pyautogui, watchdog, pdfplumber, pdfrw, spacy, pandas, tesseract-ocr
- Chrome or Edge browser with the custom automation extension installed
- Local NLP models downloaded once: DistilBERT (summarization), spaCy en_core_web_sm (classification)
- LibreTranslate local instance (optional, for clients handling non-English callers)
- Local config folder with JSON files: field mappings, templates, routing rules, carrier fax directory
- Network access only to systems agents already use — no new endpoints or firewall rules needed

Risk Register

Risk	Likelihood	Impact	Mitigation
Portal UI changes break DOM scrapers	High	Medium	Modular scraper profiles per portal; auto-detect breakage and alert; quick config update process
CTM/SF DOM structure differs per client	Medium	High	Client-specific selector config files; one-time mapping session per deployment
Local NLP model accuracy below threshold	Medium	Medium	Confidence threshold gates; low-confidence outputs shown as drafts, not auto-posted
Agent machine performance impacted by	Low	Medium	Lightweight polling intervals; scripts idle when agent is not on a call

Risk	Likelihood	Impact	Mitigation
background scripts			
PDF form structure varies across insurers	High	Medium	Pdfplumber field-label matching; fallback to OCR for non-standard PDFs; manual override always available

Estimated Time Savings Summary

Client	Key Automation	Time Lost Currently	Projected Savings
Flyland Recovery	CTM-SF session pop-up + auto-notes	1–3 hrs/shift	~60–90 min/shift
Legacy	Multi-tab wrap-up sync + AI prioritizer	1–2 hrs/shift	~60–90 min/shift
TBT	CSV-based lead pruning + auto-dialer	Half shift on dead leads	~2–3 hrs/shift
Takahami	Portal RPA fax + status DOM updater	20 min–2 hrs/shift	~60–90 min/shift
Banyan	Session pop-up + multi-tab auto-tracking	40% shift on manual entry	~2 hrs/shift
Element Medical	PDF auto-filler + portal scraper	50% shift in queue + 10+ forms	~2–3 hrs/shift